1. Getting Started

Python

```
import pandas as pd

import numpy as np
```

2. Creating & Viewing Data

Command	Description
pd.DataFrame(data)	Create a DataFrame from a dictionary or list.
pd.read_csv('file.csv')	Load data from a CSV file.
df.head(n)	View the first *n* rows (default 5).
df.tail(n)	View the last *n* rows.
df.info()	Get summary of data types and non-null counts.
df.describe()	Get statistical summary (mean, min, max) of numeric columns.
df.shape	Get number of rows and columns (rows, cols).
df.columns	Get a list of all column names.

---

3. Selection & Slicing (The "How do I grab data?" Guide)

A. Column Selection

Python

```
df['ColumnName']       # Select one column (returns Series)

df[['Col1', 'Col2']]    # Select multiple columns (returns DataFrame)
```

B. Row Selection (loc vs iloc)

Feature	.loc (Label Based)	.iloc (Position Based)
Uses	Row/Column **Names** (Strings)	Row/Column **Indices** (Integers 0, 1...)
Slicing	**Inclusive** (Start:End includes End)	**Exclusive** (Start:End excludes End)
Syntax	df.loc[row_label, col_label]	df.iloc[row_pos, col_pos]

Examples:

Python

```
# Select Row "ID_100" and Column "Name"
df.loc['ID_100', 'Name']

# Select First 5 rows and First 3 columns
df.iloc[0:5, 0:3]
```

C. Conditional Filtering (Boolean Indexing)

Python

```
# Filter: Math Score > 80
df[df['Math'] > 80]

# Filter: AND Condition (Use &)
df[(df['Math'] > 80) & (df['Class'] == 'A')]

# Filter: OR Condition (Use |)
df[(df['Math'] > 80) | (df['Science'] > 90)]
```

---

4. Handling Dirty Data

A. Missing Data (NaN)

Command	**Description**
df.isnull().sum()	Count missing values in each column.
df.dropna()	Drop rows that have *any* missing values.
df.fillna(value)	Replace missing values with a specific value (e.g., 0).
df.fillna(df.mean())	Replace missing values with the average (mean).

B. Duplicates

Python

```
# Check for duplicates
df.duplicated().sum()

# Drop duplicates (keep first instance)
df.drop_duplicates(keep='first', inplace=True)
```

---

5. Combining Datasets

A. Concatenation (Stacking)

Use when you have data split into parts (e.g., January Data + February Data)

Python

```
# Stack vertically (Row on Row)
pd.concat([df1, df2], axis=0)

# Stack horizontally (Side by Side)
pd.concat([df1, df2], axis=1)
```

B. Merging (SQL Joins)

Use when you need to link tables by a common ID.

Python

```
# Inner Join (Only matching records)
pd.merge(df1, df2, on='Student_ID', how='inner')

# Left Join (Keep all df1, match df2 where possible)
pd.merge(df1, df2, on='Student_ID', how='left')
```

---

6. Modifying Data

Python

```
# Adding a new column (Math + Science)
df['Total'] = df['Math'] + df['Science']

# Deleting a column
df.drop(columns=['UnwantedColumn'], inplace=True)

# Sorting values (High to Low)
df.sort_values(by='Score', ascending=False)
```